HARNESS LAB · MÓDULO 3 · AULA 3.3

Skills: *conhecimento sob demanda*

O que vale sempre fica no CLAUDE.md. O que vale às vezes espera ser chamado. E quem chama é o modelo, lendo só o nome e a descrição.

`SKILL.md` `description` `when_to_use` `disable-model-invocation` `user-invocable` `allowed-tools` `paths` `context: fork` `nova-migration`

VOCÊ SAI COM

A `nova-migration` no seu projeto, disparando sozinha pelo pedido e terminando numa prova (`migrate`, `rollback`, `migrate`). E o plugin `skill-gatilho`, que escreve a descrição no padrão da Anthropic e confere se o seu leque de skills está enxuto.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

CLAUDE.md e skill servem para a mesma coisa: dar contexto e regra ao agente. A diferença é quando entram. O CLAUDE.md carrega no começo da sessão e vai junto em toda mensagem, queira você ou não. Por isso ele tem que ser pequeno. A skill é o modelo que decide usar: ele vê sempre só um catálogo com o nome e a descrição de cada uma, e carrega o resto sob demanda. Por isso a skill pode ser mais detalhada.



Como o nome e a descrição são tudo o que o modelo vê, quanto melhor a descrição, melhor a decisão de usar ou não. A dica da aula: use o padrão de escrita de descrição da Anthropic e peça ao Claude Code para escrever a descrição com essas regras. A skill mora em `.claude/skills/<nome>/SKILL.md` (outros agentes usam `.agents/skills`), vai para o git e o time inteiro recebe.

Vira skill o que você ensina repetido: o trecho da documentação do Vue que você cola toda vez que pede uma feature de front. Mas poucas skills. O modelo pode decidir usar várias de uma vez, e tudo isso entra no contexto. Com 40 skills já é demais, e ponteiro para skill dentro do CLAUDE.md está errado: o modelo decide sozinho, pelo nome e pela descrição. "Ajuda com banco de dados e boas práticas de migration" não deixa o modelo decidir, e ele pode decidir não usar. O `when_to_use` é o campo para dizer quando.

Criar

`nova-migration`, revisar PR, integração interna com regra de negócio: procedimento da casa que você repete.

Não criar

Persona de "dev sênior", doc de framework (o Boost e o context7 já trazem), skill de uma vez só e as gêmeas (review, revisar, checar PR: junte numa).

Sempre não é skill

Regra em texto que vale sempre vai no CLAUDE.md. Se alguém morre quando o modelo esquece, é hook.

Skill que se paga

Descrição com gatilho, passos curtos com o comando exato, o porquê dos passos e uma checagem no fim, que prova que o que ela ensinou foi feito.

No frontmatter, `disable-model-invocation` tira a skill do catálogo do modelo (só você chama) e `user-invocable: false` faz o contrário. Também dá para limitar por caminho (`paths`), liberar ferramentas (`allowed-tools`) e rodar em contexto próprio (`context: fork`). No fim, o Laravel Boost em ação: num blog com bastante feature de IA, ele instalou cinco skills. A `ai-sdk-development` só entra quando a tarefa mexe com o SDK de IA do Laravel, e a descrição diz isso. A aula não traz um "Boostify de skills": para cada stack já existe repositório de boas skills. Pegue de 4 a 5, no máximo 10.

> *"Create a skill when you keep pasting the same instructions, checklist, or multi-step procedure into chat, or when a section of CLAUDE.md has grown into a procedure rather than a fact."*
>
> Documentação oficial do Claude Code · Extend Claude with skills

2 O que a aula *não teve tempo de dizer*

ALÉM DA AULA · AS REGRAS QUE A DICA DA AULA MANDA SEGUIR

O "padrão de escrita de descrição da Anthropic" está no guia *Skill authoring best practices*. Em resumo: diga **o que a skill faz e quando usar**; escreva na **terceira pessoa** ("Cria...", nunca "Eu posso ajudar"), porque a descrição entra no prompt de sistema; seja específico, com as palavras que alguém usaria no pedido. O nome: só minúsculas, números e hífen, até 64 caracteres, sem "claude" nem "anthropic". No Claude Code, `description` e `when_to_use` juntos são cortados em 1.536 caracteres na lista: ponha o uso principal primeiro. Se a skill for para o claude.ai ou para a API, o `when_to_use` não é aceito e a descrição tem limite de 1.024.

Fontes: Skill authoring best practices, "Writing effective descriptions" e "YAML frontmatter requirements"; Skills, "Frontmatter reference" e "Using skill frontmatter outside Claude Code".

ALÉM DA AULA · A DESCRIÇÃO RUIM ÀS VEZES DISPARA, POR SORTE

Teste feito para este material, em outro projeto, um Laravel 13 novo com o Boost 2.10.3, que instala seis skills (uma delas, `laravel-best-practices`, também fala de migration) e o pedido da aula, "adiciona a coluna cpf em clientes", em sessão nova, sem citar a skill. Com a descrição da `nova-migration` deste repositório, o modelo chamou a skill como primeira ação nas quatro rodadas. Trocando por "Ajuda com banco de dados e boas práticas de migration.", uma rodada não chamou e as outras duas só chamaram depois de vasculhar arquivos. São poucas rodadas, não estatística, mas é o padrão que a aula descreveu. Com a skill chamada, ele seguiu os passos: olhou o tamanho da tabela, conferiu o que já tinha rodado, fez a coluna nullable e só deu por terminado depois do `prova: ok`.

ALÉM DA AULA · O CATÁLOGO TEM ORÇAMENTO, E SKILL QUE SÓ VOCÊ CHAMA NÃO PAGA

As descrições dividem um espaço de 1% da janela de contexto. Quando estoura, o Claude Code tira primeiro a descrição das skills que você menos usa: o nome fica, as palavras que o modelo usaria para escolher somem. Com `disable-model-invocation: true` a descrição nem entra no contexto, e a skill continua no `/`. É o lugar da skill de release, de deploy e da que você quer guardar. Para ver o custo: a linha Skills do `/context`, o `/doctor` e o `/skill-doctor`, que lista as skills nunca chamadas. Para desligar sem apagar o arquivo do time: no `/skills`, Espaço alterna o estado (`on`, `name-only`, `user-invocable-only`, `off`) e Esc grava em `.claude/settings.local.json`, que é só seu (exemplo em `modelos/settings.local.skill-overrides.json`).

Fonte: Skills, "Skill descriptions are cut short", "Control who invokes a skill", "Override skill visibility from settings" e "Find unused skills".

ALÉM DA AULA · SKILL DE UMA VEZ SÓ, SEM ENTRAR NO PROJETO

O "roda sem colocar no projeto" da aula tem dois caminhos. Deixe a skill numa pasta fora do repositório, em `~/skills-avulsas/.claude/skills/`, e abra a sessão com `claude --add-dir ~/skills-avulsas`: o Claude Code carrega as skills dessa pasta só naquela sessão. Ou use `~/.claude/skills/`, que vale em todos os seus projetos e não vai para o git (e ganha da skill do projeto quando o nome é igual). Usou e terminou a migração para a versão nova? Apague a pasta.

Fonte: Skills, "Choose where skills load", "Load skills from a directory outside the project" e "Resolve skills that share a name".

ALÉM DA AULA · CARREGOU UMA VEZ, FICA. E NA COMPACTAÇÃO, SÓ O COMEÇO

Quando a skill é chamada, o corpo entra na conversa uma vez e fica. O Claude Code não relê o arquivo nas voltas seguintes. Por isso a `nova-migration` abre com "estes passos valem para a tarefa inteira": instrução de passo único some da atenção depois do primeiro uso. Na compactação, cada skill volta com no máximo os primeiros 5.000 tokens, 25.000 no total, e as chamadas há mais tempo saem primeiro. O importante vai no topo. E a regra que tem que valer toda vez, como "nunca edite migration de produção", é hook (`PreToolUse` em `database/migrations`, da aula 2.3), não linha de skill: o modelo pode nem chamar a skill.

Fonte: Skills, "Skill content lifecycle" e "Claude stops following a skill".

ALÉM DA AULA · O QUE O VALIDATE NÃO PEGA, E O BOOST

`claude plugin validate .claude/skills` pega YAML quebrado (a skill carregaria sem nenhum campo) e descrição faltando. Teste feito para este material: ele aprovou uma descrição de 1.700 caracteres e não reclamou de `when-to-use` com hífen, campo que o Claude Code ignora sem avisar. Frontmatter que não começa na primeira linha vira conteúdo. Por isso o `confere.py` do plugin `skill-gatilho`. Em projeto com Laravel Boost, escreva a skill do time em `.ai/skills/<nome>/`: o `boost:update` põe em `.claude/skills` e nos outros agentes configurados. Pelo código do Boost 2.10, ele só remove as skills que ele mesmo instalou, e uma skill sua com o nome de uma dele substitui a dele. Ah, o `db:table` do `SKILL.md` que aparece na tela da aula mostra o tamanho da tabela, não as linhas: quem conta linhas é o `db:show --counts`.

Fontes: Skills, "Frontmatter reference" e "Skill not triggering"; Laravel Boost, "Custom Skills" e "Overriding Skills"; código de laravel/boost 2.10.3 e laravel/framework 13.

3 Instale *no seu projeto*

Vai para o seu repositório: a skill `.claude/skills/nova-migration/` (`SKILL.md`, `scripts/prova.sh`, `references/stacks.md`). Fica fora dele: o plugin `skill-gatilho`. Nada seu é apagado.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale a skill no seu projeto.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 3.3
```

A saída mostra `criado` para os três arquivos. Um `conflito`: você já tinha uma `nova-migration`. O seu `SKILL.md` fica, o da aula vira `SKILL.md.harness-lab-novo` e o `prova.sh` e o `stacks.md` entram na sua pasta. Junte antes de testar.

3 Confira se o frontmatter é válido.

Terminal · na raiz do seu projeto — claude

```
claude plugin validate .claude/skills
```

O esperado é `Validation passed`. Ele confere todas as skills da pasta, as suas também.

4 Teste a prova fora do Claude Code.

O `prova.sh` é a checagem do fim da skill. Com `--seco` ele só mostra a stack que achou e o que rodaria:

Terminal · na raiz do seu projeto — shell

```
.claude/skills/nova-migration/scripts/prova.sh . --seco
```

o que aparece num projeto Laravel com Sail — saída do --seco

```
prova: Laravel (Sail) · /home/voce/loja
  rodaria  ./vendor/bin/sail artisan migrate --no-interaction
  rodaria  ./vendor/bin/sail artisan migrate:rollback --step=1 --no-interaction
  rodaria  ./vendor/bin/sail artisan migrate --no-interaction
prova: seco (nada foi rodado)
```

Reconhece Laravel (com ou sem Sail), Rails, Alembic, Prisma e Django, recusa `APP_ENV=production` e nunca passa `--force`. Sem `--seco` roda de verdade, no banco de desenvolvimento.

5 **Instale o plugin do curso.**

Terminal · qualquer pasta — `claude`

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin marketplace update harness-lab
claude plugin install skill-gatilho@harness-lab
```

Prefere de dentro do Claude Code? É o mesmo, e o `/reload-plugins` carrega sem reiniciar:

Dentro do Claude Code — `sessão`

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin marketplace update harness-lab
/plugin install skill-gatilho@harness-lab
/reload-plugins
```

6 **Faça o teste da aula, numa sessão nova.**

Sessão nova importa: o contexto de quem acabou de mexer na skill esconde as falhas da descrição. Confira a lista e faça o pedido sem citar a skill. Não tem tabela `clientes`? Troque por uma tabela sua.

Dentro do Claude Code · sessão nova — `sessão`

```
/skills
> adiciona a coluna cpf em clientes
```

O esperado: a `nova-migration` sendo chamada logo no começo, o tamanho da tabela conferido, a migration nova e o `prova: ok` antes do "terminei". O `prova.sh` roda sem pedir permissão porque o `allowed-tools` da skill libera só ele, só naquela volta.

7 **Peça a descrição com as regras.**

A dica da aula, empacotada. A skill do plugin roda o conferidor, reescreve a `description` e o `when_to_use`, mostra antes e depois e só grava com o seu ok:

Dentro do Claude Code — `sessão`

```
/skill-gatilho:descrever nova-migration
```

Sem o plugin, o pedido completo está em `modelos/pedir-a-descricao.md`. O conferidor também roda sozinho, direto da pasta que você clonou:

Terminal · na raiz do seu projeto — `shell`

```
S=~/harness-lab-materiais/plugins/skill-gatilho/skills/descrever
python3 -I "$S/scripts/confere.py" .claude/skills
```

8 Escreva a sua primeira skill.

A aula recomenda escrever à mão, a partir de um procedimento seu (a seção que saiu do CLAUDE.md na 3.1 é um bom começo). O esqueleto tem as quatro partes de uma skill que se paga:

Terminal · na raiz do seu projeto — shell

```
mkdir -p .claude/skills/minha-skill
M=$(echo ~/harness-lab-materiais/modulo-3-regra/aula-3.3-*/modelos)
cp "$M/primeira-skill.md" .claude/skills/minha-skill/SKILL.md
```

O `disable-model-invocation: true` do esqueleto deixa o rascunho fora do catálogo do modelo. Troque o nome da pasta e o `name`, preencha, apague essa linha e teste igual ao passo 6. A pasta `.claude/skills` não existia quando a sessão abriu? Rode `/reload-skills`.

9 Skill de uma vez só, fora do projeto (opcional).

Terminal · na sua pasta pessoal — shell

```
mkdir -p ~/skills-avulsas/.claude/skills
```

Ponha a skill lá dentro e abra a sessão, na raiz do projeto, enxergando essa pasta:

Terminal · na raiz do seu projeto — claude

```
claude --add-dir ~/skills-avulsas
```

10 Projeto com Laravel Boost (não roda na sandbox).

Terminal · projeto Laravel com Boost — laravel

```
mkdir -p .ai/skills
cp -r .claude/skills/nova-migration .ai/skills/
php artisan boost:update
```

O arquivo que o instalador colocou

`.claude/skills/nova-migration/SKILL.md` · resumido — instalado pelo instalar.sh · não copie do PDF

```
---
name: nova-migration
description: Cria ou altera tabela e coluna neste projeto sem travar
  produção (coluna nova nullable, backfill em job, NOT NULL numa ...) ...
when_to_use: Pedido de migration, coluna nova, ... "adiciona a coluna cpf ..."
allowed-tools: Bash(${CLAUDE_SKILL_DIR}/scripts/prova.sh *)
---
# Nova migration
Estes passos valem para a tarefa inteira, não só para a primeira migration.
1. Veja se a tabela é grande: db:show --counts (linhas), db:table (tamanho).
2. Tabela grande: nullable, backfill num job, NOT NULL numa segunda migration.
   Por quê: um UPDATE de milhões de linhas segura a tabela e o deploy.
3. Índice em tabela grande: migration só dele.
4. Nunca edite migration que já rodou em produção. migrate:status mostra.
5. Uma mudança por migration, com down() que desfaz o up().
6. Prove: ${CLAUDE_SKILL_DIR}/scripts/prova.sh . (não precisa ler o script)
```

Repare nas quatro partes da aula. A descrição diz o que faz e o detalhe que diferencia; o `when_to_use` traz as palavras do pedido. Os passos têm o comando exato. O porquê aparece só no passo que parece exagero. E o fim é uma checagem que a skill roda, em vez de pedir para o modelo "conferir". O `${CLAUDE_SKILL_DIR}` vira a pasta da skill, esteja ela no projeto, em `~/.claude/skills` ou num plugin. Stack neutra: em Rails a prova é `db:migrate`, `db:rollback STEP=1`, `db:migrate`; em Alembic, `upgrade head`, `downgrade -1`, `upgrade head`; em Prisma, que não tem rollback automático, `migrate deploy` e `migrate status`.

4 Arquivos que *reforçam a aula*

`.claude/skills/nova-migration/`

A skill que a aula citou como "um tipo legal", testada de ponta a ponta num Laravel 13, com os três níveis: o `SKILL.md`, o `scripts/prova.sh` (roda, só o resumo entra) e o `references/stacks.md` (lido só fora do Laravel).

No repositório: modulo-3-regra/aula-3.3-skills-conhecimento-sob-demanda/projeto/

`plugins/skill-gatilho`

A dica de pedir a descrição com as regras, como skill de uso geral, e o `confere.py`: corte de 1.536, campo com nome errado, nome fora do padrão, skills gêmeas, leque com mais de 10.

No repositório: plugins/skill-gatilho/skills/descrever/

`modelos/pedir-a-descricao.md`

O pedido para colar no Claude Code, com as regras da Anthropic, e o pedido para transformar um procedimento seu numa skill inteira.

No repositório: modulo-3-regra/aula-3.3-skills-conhecimento-sob-demanda/modelos/

`modelos/primeira-skill.md`

O esqueleto da primeira skill com as quatro partes: descrição com gatilho, passos com comando exato, o porquê e a checagem.

No repositório: modulo-3-regra/aula-3.3-skills-conhecimento-sob-demanda/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
A skill não aparece no `/skills`	A pasta `.claude/skills` foi criada com a sessão aberta, ou a skill está numa subpasta onde o Claude ainda não mexeu	`/reload-skills`. Skill de subpasta carrega quando o Claude lê ou edita um arquivo de lá
Aparece, mas o modelo nunca chama	Descrição que diz o tema e não diz quando, ou YAML quebrado: a skill carrega sem campo nenhum	`claude plugin validate .claude/skills`, depois `/skill-gatilho:descrever`. Teste numa sessão nova
`when-to-use` ou `disable_model_invocation` sem efeito	Nome de campo errado é ignorado sem aviso, e o validate não reclama	Os nomes exatos: `when_to_use` com sublinhado, o resto com hífen. O `confere.py` aponta

SINTOMA	CAUSA	CORREÇÃO
O fim da descrição não chega ao modelo	`description` + `when_to_use` acima de 1.536 caracteres, ou o catálogo estourou o orçamento	Uso principal primeiro, texto mais curto. `/skill-doctor` mostra as skills sem uso para desligar
`user-invocable: false` e o modelo continua chamando	Esse campo só tira do menu `/`. Quem tira do modelo é o outro	`disable-model-invocation: true`
Pus `allowed-tools: Read` e ela editou arquivo	`allowed-tools` só libera sem pedir, não restringe nada	Para bloquear: `disallowed-tools`, regra de permissão ou hook
O teste com `claude -p` diz que a skill falhou	Testado aqui: no `-p`, a chamada da ferramenta Skill foi negada até ela entrar na lista de liberadas	`--allowedTools Skill` no teste, ou teste na sessão interativa

6 Você sai *com*

- [ ] `.claude/skills/nova-migration/` no seu projeto, com `claude plugin validate .claude/skills` passando.
- [ ] O `prova.sh . --seco` reconhecendo a sua stack.
- [ ] Numa sessão nova, o pedido de coluna nova chamando a `nova-migration` sem você citar, e terminando em `prova: ok`.
- [ ] O plugin `skill-gatilho` instalado e o `confere.py` sem ERRO nas suas skills.
- [ ] A sua primeira skill escrita, com descrição, passos curtos, porquê e checagem, testada numa sessão nova.
- [ ] Uma skill que você não usa desligada ou apagada.

PROMETIDO NA AULA

- ✓ "Peça para o seu Claude Code escrever a descrição da skill com essas regras": o pedido pronto em `modelos/pedir-a-descricao.md` e a skill `/skill-gatilho:descrever` (seção 3, passo 7). As regras estão na seção 2.
- ✓ "Isso entra nesse when_to_use": o campo na `nova-migration` e em `references/regras.md` do plugin, com o limite de 1.536 caracteres.
- ✓ "Nova Migration é um tipo legal": a skill testada em `projeto/.claude/skills/nova-migration/`, instalada com `instalar.sh 3.3` (passo 2).
- ✓ "Que ela também ensine uma forma de validar": o `scripts/prova.sh`, que fecha a skill com migrate, rollback e migrate (passo 4).
- ✓ "Criasse na mão a sua primeira skill": o esqueleto `modelos/primeira-skill.md` (passo 8). Travou? Leve o procedimento e a saída do `confere.py` para o chat da aula.
- ✓ "Roda essa skill sem colocar no projeto": `--add-dir` com uma pasta fora do repositório (seção 2 e passo 9).
- ✓ "Vou mostrar o Laravel Boost em ação": as skills do Boost, o `.ai/skills` e o `boost:update` (seção 2 e passo 10).

✓ "Já existe repositórios de boas skills": os catálogos oficiais na seção 8.

7 Glossário

Skill	Pasta com um `SKILL.md`: conhecimento ou procedimento que entra no contexto só quando é usado.
Catálogo	A lista de nomes e descrições que o modelo vê desde o início. Tem orçamento: 1% da janela de contexto.
description	O que a skill faz e quando usar. É o gatilho: o modelo escolhe por ela.
when_to_use	Campo do Claude Code para as frases de pedido. Soma com a description no corte de 1.536 caracteres.
disable-model-invocation	Só você chama, com `/nome`. A descrição sai do catálogo.
user-invocable: false	Some do menu `/`. O modelo continua podendo chamar.
allowed-tools	Ferramentas liberadas sem pedir permissão na volta em que a skill é chamada. Não restringe.
context: fork	A skill roda num subagente, com contexto próprio. Assunto da 3.4.

8 Para *ir além*

- Extend Claude with skills: todos os campos do frontmatter, onde cada skill carrega, o ciclo de vida do conteúdo e a seção "Skill not triggering".
- Skill authoring best practices: o padrão de descrição da dica da aula, nomes, scripts e a lista de conferência antes de compartilhar.
- anthropics/skills e o marketplace `claude-plugins-official`: skills de referência da Anthropic. Lá está o plugin `skill-creator`, que gera pedidos que devem e que não devem disparar a skill e mede o acerto da descrição.
- Laravel Boost: "Agent Skills", "Custom Skills" e "Overriding Skills". No Boost 2.10, `php artisan boost:add-skill` traz skills de um repositório do GitHub.
- Agent Skills: o padrão aberto que o Claude Code segue, e quais campos funcionam fora dele.

Próxima: *Aula 3.4 · Subagente: quando usar e quando é foguete que dá ré*

A skill traz conhecimento para dentro do contexto. O `context: fork` já apontou o outro lado: e quando você quer tirar trabalho de dentro do contexto?